About

Project Name:"Crop Yield Prediction Using Machine Learning"

Data loading

Subtask:

Load the dataset "Crop_recommendation.csv" into a pandas DataFrame.

```
import pandas as pd
from google.colab import drive
from google.colab import files
drive.mount('/content/drive')
uploaded = files.upload()

try:
    df = pd.read_csv('/content/Crop_recommendation.csv')
    display(df.head())
    print()
    print("Shape of the DataFrame:")
    print(df.shape)
except FileNotFoundError:
    print("Error: 'Crop_recommendation.csv' not found.")
    df = None
except Exception as e:
    print(f"An error occurred: {e}")
    df = None
```

```
Mounted at /content/drive
```

Choose Files Crop_reco…endation.csv

Crop_recommendation.csv(text/csv) - 144837 bytes, last modified: 5/16/2025 - 100% done

```
Saving Crop_recommendation.csv to Crop_recommendation.csv
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
Shape of the DataFrame:
(2200, 8)
```

Data exploration

Subtask:

Explore the loaded dataset to understand its characteristics.

```
# Data Types and Shape
print("Data Types:\n", df.dtypes)
print("\nShape:", df.shape)


# Missing Values
print("\nMissing Values:\n", df.isnull().sum())

# Descriptive Statistics for Numerical Features
numerical_features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
print("\nDescriptive Statistics for Numerical Features:\n", df[numerical_features].describe())

# Unique Values and Frequencies for Categorical Features
```

```
categorical_features = ['label']
for col in categorical_features:
    print(f"\nUnique values in '{col}':\n{df[col].unique()}")
    print(f"\nFrequency of each category in '{col}':\n{df[col].value_counts()}")
```

```
humidity       0
ph             0
rainfall       0
label          0
dtype: int64

Descriptive Statistics for Numerical Features:
                 N            P            K  temperature     humidity  \
count  2200.000000  2200.000000  2200.000000  2200.000000  2200.000000
mean     50.551818    53.362727    48.149091    25.616244    71.481779
std      36.917334    32.985883    50.647931     5.063749    22.263812
min       0.000000     5.000000     5.000000     8.825675    14.258040
25%      21.000000    28.000000    20.000000    22.769375    60.261953
50%      37.000000    51.000000    32.000000    25.598693    80.473146
75%      84.250000    68.000000    49.000000    28.561654    89.948771
max     140.000000   145.000000   205.000000    43.675493    99.981876

                ph     rainfall
count  2200.000000  2200.000000
mean      6.469480   103.463655
std       0.773938    54.958389
min       3.504752    20.211267
25%       5.971693    64.551686
50%       6.425045    94.867624
75%       6.923643   124.267508
max       9.935091   298.560117

Unique values in 'label':
['rice' 'maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans'
 'mungbean' 'blackgram' 'lentil' 'pomegranate' 'banana' 'mango' 'grapes'
 'watermelon' 'muskmelon' 'apple' 'orange' 'papaya' 'coconut' 'cotton'
 'jute' 'coffee']

Frequency of each category in 'label':
label
rice           100
maize          100
chickpea       100
kidneybeans    100
pigeonpeas     100
mothbeans      100
mungbean       100
blackgram      100
lentil         100
pomegranate    100
banana         100
mango          100
grapes         100
watermelon     100
muskmelon      100
apple          100
orange         100
papaya         100
coconut        100
cotton         100
jute           100
coffee         100
Name: count, dtype: int64
```

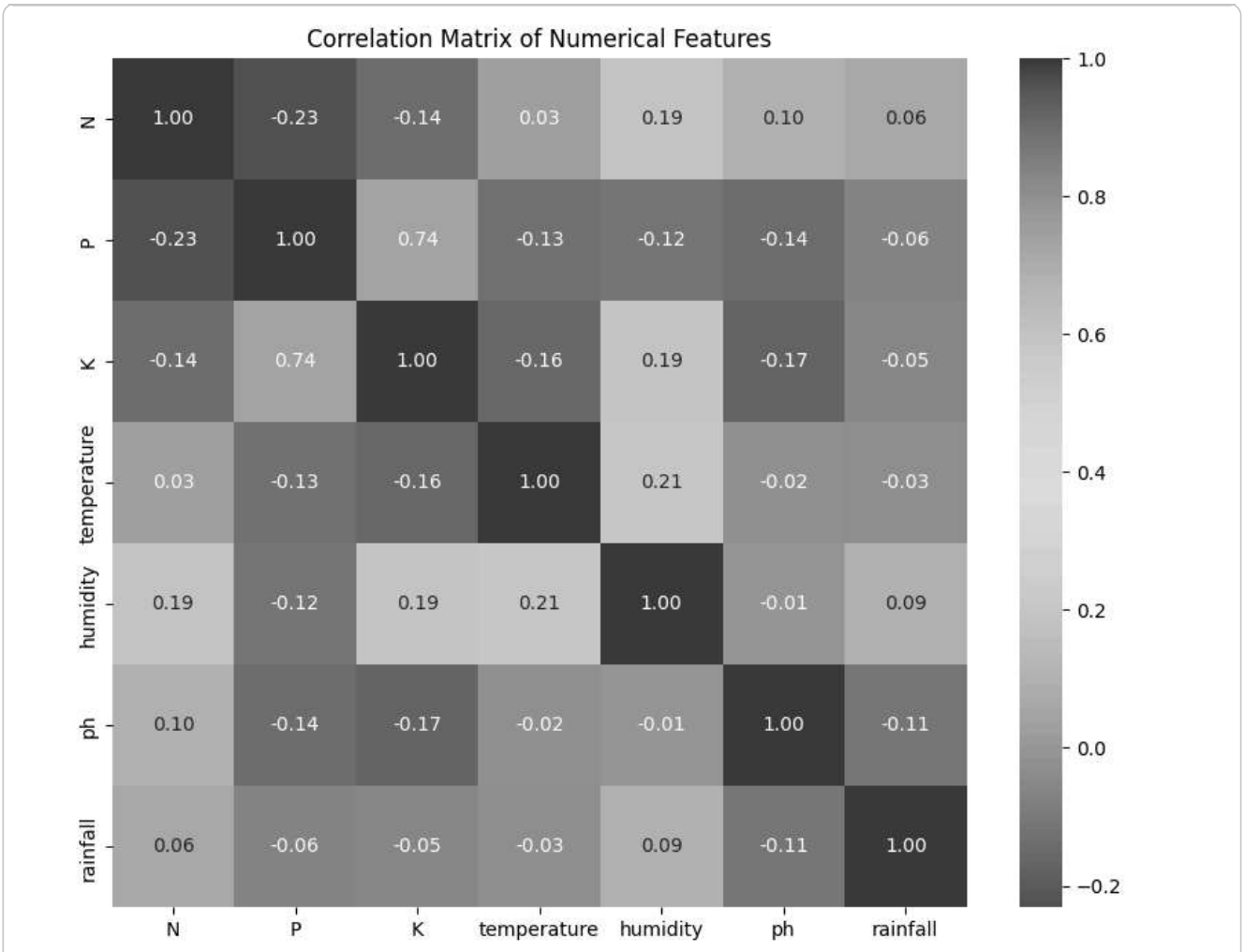
Data analysis

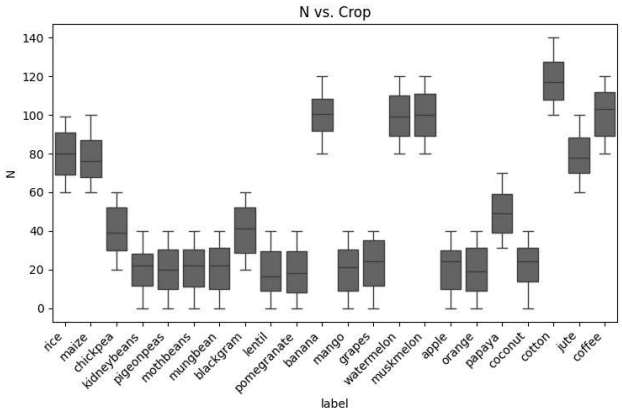
Subtask:

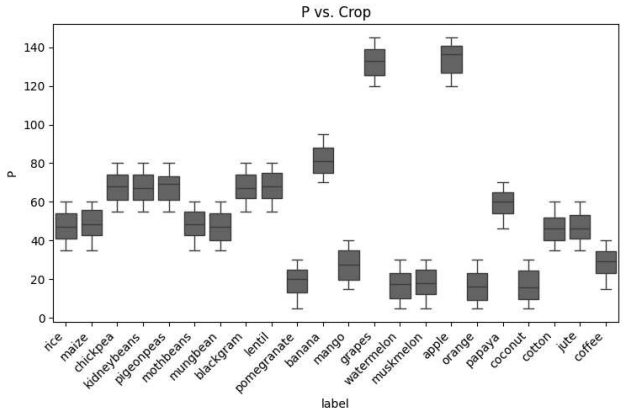
Analyze the relationships between different features in the dataset, with a focus on the relationship between input features and the 'label' (crop).

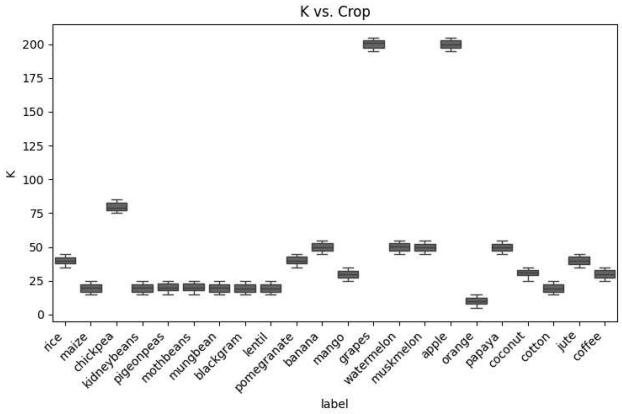
```
import matplotlib.pyplot as plt
import seaborn as sns
import seaborn as sns

df = pd.read_csv('/content/Crop_recommendation.csv')
```

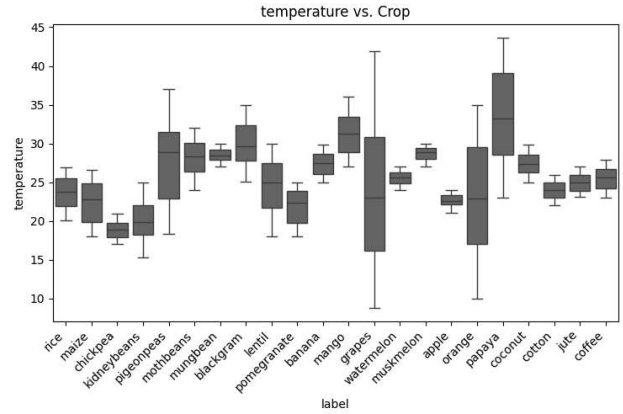
```
# Calculate the correlation matrix
correlation_matrix = df.corr(numeric_only=True)
# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()

# Box plots of numerical features for each crop type
numerical_features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
plt.figure(figsize=(15, 20))
for i, feature in enumerate(numerical_features):
    plt.subplot(4, 2, i + 1)
    sns.boxplot(x='label', y=feature, data=df)
    plt.title(f'{feature} vs. Crop')
    plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

plt.figure(figsize=(20, 12))
plt.subplot(2, 2, 1)
sns.scatterplot(data=df, x='N', y='P', hue='label', palette='viridis', alpha=0.6)
plt.title('Nitrogen vs Phosphorous Content')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Temperature vs Humidity
plt.subplot(2, 2, 2)
sns.scatterplot(data=df, x='temperature', y='humidity', hue='label', palette='viridis', alpha=0.6)
plt.title('Temperature vs Humidity')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# pH vs Rainfall
plt.subplot(2, 2, 3)
sns.scatterplot(data=df, x='ph', y='rainfall', hue='label', palette='viridis', alpha=0.6)
plt.title('pH vs Rainfall')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# K vs Temperature
plt.subplot(2, 2, 4)
sns.scatterplot(data=df, x='K', y='temperature', hue='label', palette='viridis', alpha=0.6)
plt.title('Potassium vs Temperature')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()

# Calculate the mean value of each numerical feature for each crop
crop_means = df.groupby('label')[numerical_features].mean()
display(crop_means)

# Summarize observations
print("Observations:")
print("The correlation matrix heatmap shows the linear relationships between numerical features. The box plots
```

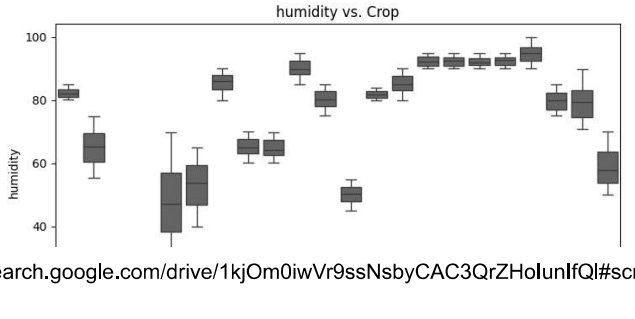

Correlation Matrix of Numerical Features
N
P
K
temperature
humidity
ph
rainfall
1.00 -0.23 -0.14 0.03 0.19 0.10 0.06
-0.23 1.00 0.74 -0.13 -0.12 -0.14 -0.06
-0.14 0.74 1.00 -0.16 0.19 -0.17 -0.05
0.03 -0.13 -0.16 1.00 0.21 -0.02 -0.03
0.19 -0.12 0.19 0.21 1.00 -0.01 0.09
0.10 -0.14 -0.17 -0.02 -0.01 1.00 -0.11
0.06 -0.06 -0.05 -0.03 0.09 -0.11 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

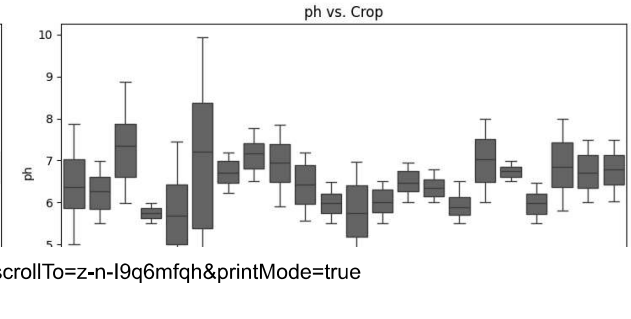

N vs. Crop
N
label
rice maize chickpea kidneybeans pigeonpeas mothbeans mungbean blackgram lentil pomegranate banana mango grapes watermelon muskmelon apple orange papaya coconut cotton jute coffee

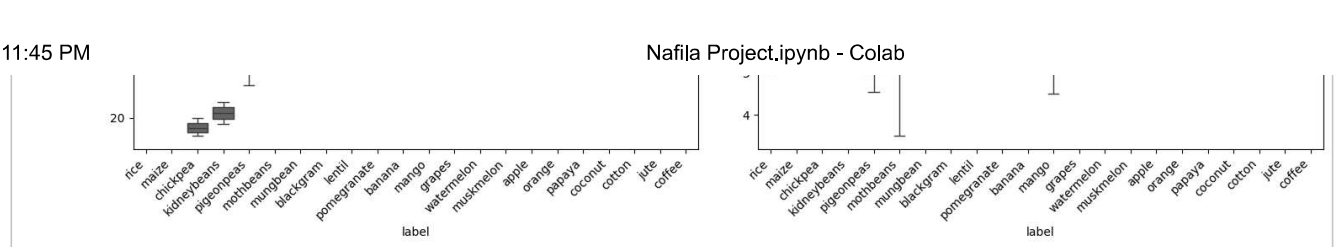

P vs. Crop
P
label
rice maize chickpea kidneybeans pigeonpeas mothbeans mungbean blackgram lentil pomegranate banana mango grapes watermelon muskmelon apple orange papaya coconut cotton jute coffee

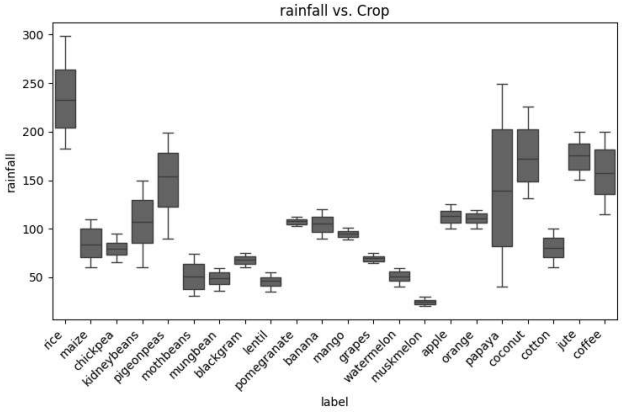

K vs. Crop
K
label
rice maize chickpea kidneybeans pigeonpeas mothbeans mungbean blackgram lentil pomegranate banana mango grapes watermelon muskmelon apple orange papaya coconut cotton jute coffee

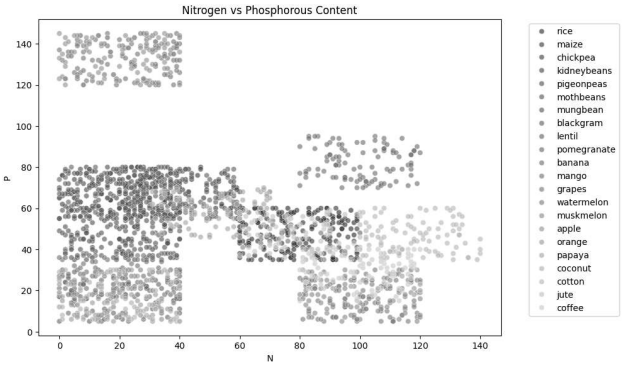

temperature vs. Crop
temperature
label
rice maize chickpea kidneybeans pigeonpeas mothbeans mungbean blackgram lentil pomegranate banana mango grapes watermelon muskmelon apple orange papaya coconut cotton jute coffee

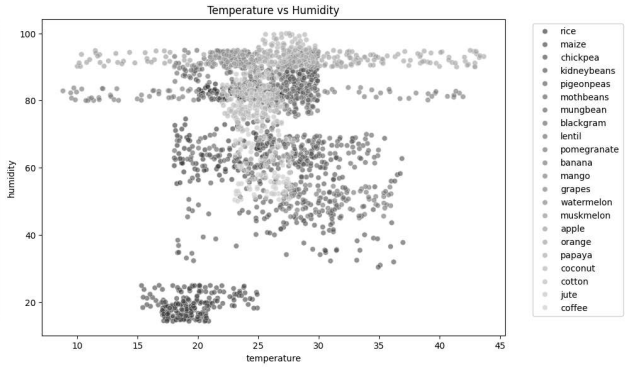

humidity vs. Crop
humidity

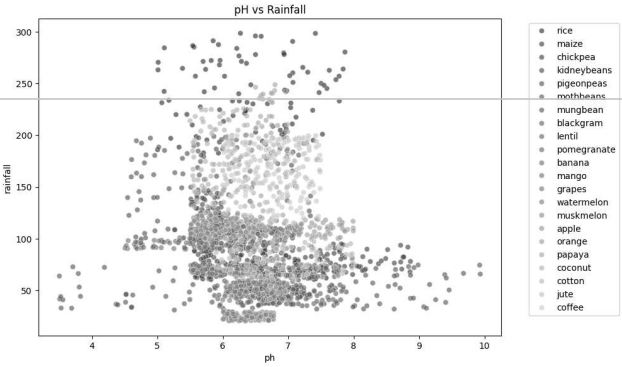

ph vs. Crop
ph

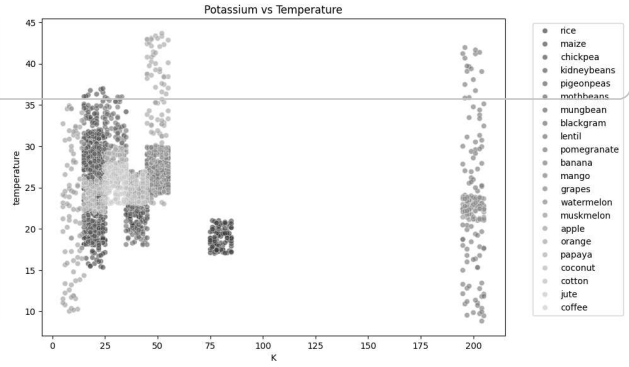












label	N	P	K	temperature	humidity	ph	rainfall
apple	20.80	134.22	199.89	22.630942	92.333383	5.929663	112.654779
banana	100.23	82.01	50.05	27.376798	80.358123	5.983893	104.626980
blackgram	40.02	67.47	19.24	29.973340	65.118426	7.133952	67.884151
chickpea	40.09	67.79	79.92	18.872847	16.860439	7.336957	80.058977
coconut	21.98	16.93	30.59	27.409892	94.844272	5.976562	175.686646
coffee	101.20	28.74	29.94	25.540477	58.869846	6.790308	158.066295
cotton	117.77	46.24	19.56	23.988958	79.843474	6.912675	80.398043
grapes	23.18	132.53	200.11	23.849575	81.875228	6.025937	69.611829
jute	78.40	46.86	39.99	24.958376	79.639864	6.732778	174.792798
kidneybeans	20.75	67.54	20.05	20.115085	21.605357	5.749411	105.919778
lentil	18.77	68.36	19.41	24.509052	64.804785	6.927932	45.680454
maize	77.76	48.44	19.79	22.389204	65.092249	6.245190	84.766988
mango	20.07	27.18	29.92	31.208770	50.156573	5.766373	94.704515
mothbeans	21.44	48.01	20.23	28.194920	53.160418	6.831174	51.198487
mungbean	20.99	47.28	19.87	28.525775	85.499975	6.723957	48.403601

Next steps: Generate code with crop_means New interactive sheet

watermelon	100.32	17.72	50.08	28.663066	92.342802	6.358805	24.689952
orange	19.58	16.55	10.01	22.765725	92.170209	7.016957	110.474969

prediction

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

from tkinter import messagebox

n=float(input("Nitrogen"))
p=float(input("Phosphorus"))
k=float(input("Potassium"))
t=float(input("Temperature"))
h=float(input("Humidity"))
ph=float(input("pH"))
r=float(input("Rainfall"))

features=[]
features.append(n)
features.append(p)
features.append(k)
features.append(t)
features.append(h)
features.append(ph)
features.append(r)

df = pd.read_csv('/content/Crop_recommendation.csv')

#Train Machine Learning Models

X = df.drop('label', axis=1)
y = df['label']
class_names = np.unique(y)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# classifiers
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)

# Train models
random_forest.fit(X_train, y_train)

def evaluate(model, conf_matrix=True, return_=False):
        # Confusion Matrix
        y_pred = model.predict(X_test)
        cm = confusion_matrix(y_test, y_pred)


        # Calculating and displaying additional metrics
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='weighted')
        recall = recall_score(y_test, y_pred, average='weighted')
        f1 = f1_score(y_test, y_pred, average='weighted')

        print(f"Accuracy: {accuracy}")
        print(f"Precision: {precision}")
        print(f"Recall: {recall}")
        print(f"F1 Score: {f1}")



        # Extracting the model name for display
        model_name = str(type(model).__name__)

        # Returning metrics as a dictionary if specified
        if return_:
            metrics_dict = {
                "Model": model_name,
                "accuracy": round(accuracy, 4),
```

```
                "precision": round(precision, 4),
                "recall": round(recall, 4),
                "f1": round(f1, 4)
            }
            return metrics_dict
evaluate(random_forest)


    #capture
    # List of classifiers
classifiers = [
        random_forest
          ]

    # Initialize an empty list to store DataFrames
dfs = []

    # Loop through classifiers
for classifier in classifiers:
        # Calculate metrics for the current classifier
          metrics_result = evaluate(classifier, return_=True, conf_matrix=False)

        # Create a DataFrame from the dictionary
          metrics_df = pd.DataFrame([metrics_result])

        # Append the DataFrame to the list
          dfs.append(metrics_df)

    # Concatenate DataFrames into a single DataFrame
results_df = pd.concat(dfs, ignore_index=True)

    # Format the metrics to 4 decimal places in the DataFrame
results_df["accuracy"] = results_df["accuracy"].map("{:.4f}".format)
results_df["precision"] = results_df["precision"].map("{:.4f}".format)
results_df["recall"] = results_df["recall"].map("{:.4f}".format)
results_df["f1"] = results_df["f1"].map("{:.4f}".format)

results_df

    #Comparison of Models' Performance Metrics

results_df["precision"] = results_df["precision"].astype(float)
results_df["recall"] = results_df["recall"].astype(float)
results_df["f1"] = results_df["f1"].astype(float)



    #Make a Single Prediction Using Trained Models


    # Predict probabilities for each class
rf_probs = random_forest.predict_proba([features])[0]


    # Create a DataFrame to store the probabilities
probs_df = pd.DataFrame({
            'Class': class_names,
            'Random Forest': rf_probs,

        })

    # Melt the DataFrame for Seaborn
probs_df_melted = probs_df.melt(id_vars='Class', var_name='Model', value_name='Probability')

        # Plot using Seaborn
plt.figure(figsize=(10, 6))
sns.barplot(x='Class', y='Probability', hue='Model', data=probs_df_melted, palette='viridis')
plt.title('Predicted Probabilities for Each Class by Model', fontsize=16)
plt.xlabel('Class', fontsize=14)
plt.ylabel('Probability', fontsize=14)
plt.xticks(rotation=65)  # Rotate class names for better readability
plt.legend(title='Model', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

```
predicted_class = class_names[np.argmax(rf_probs)]
print(f"\nRecommended crop is: {predicted_class}")
```

```
Nitrogen78
Phosphorus42
Potassium42
Temperature20.13
Humidity81.60
pH7.62
Rainfall262.71
Accuracy: 0.9931818181818182
Precision: 0.9937348484848485
Recall: 0.9931818181818182
F1 Score: 0.9931754816901672
Accuracy: 0.9931818181818182
Precision: 0.9937348484848485
Recall: 0.9931818181818182
F1 Score: 0.9931754816901672
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid fe
  warnings.warn(
```

Predicted Probabilities for Each Class by Model